Function Point Analysis of InsightHive

Software Metrics

Institute of Information Technology
University of Dhaka

Supervised by:
Dr. Emon Kumar Dey
Associate Professor

Submitted by:
Hasnain Sheikh(1408)
Md. Rakibul Islam (1411)

Table of Contents

Strategy

We followed a systematic approach to Function Point Analysis by first understanding the class-based model from the SRS document. The InsightHive system presents a multi-role platform connecting gig workers, companies, and shop managers through task assignment and verification workflows. We identified function points from the client's perspective, focusing on user-visible functionality while maintaining appropriate granularity for accurate estimation.

1. About The Project

InsightHive is a comprehensive gig work management platform that facilitates connections between companies requiring task completion and gig workers seeking employment opportunities. The system incorporates shop managers for task verification and includes features for task assignment, reporting, incentive management, and performance tracking through leaderboards.

Key System Components:

- **Multi-role user management** (Gig Workers, Companies, Shop Managers, Admin)
- **Task creation and assignment workflow**
- **Real-time task verification system**
- **Comprehensive reporting with media support**
- **Incentive and rating management**
- **Performance analytics and leaderboards**
- **Notification system**

1.1 Use Case Diagram of InsightHive:

Use Case diagrams give the non-technical view of the overall system.

Use Case Modeling

Level 0:

Use Case ID: 0
Name: InsightHive App
Primary Actor: Gig Worker, Company, Shop Manager
Secondary Actor: Payment Gateway, Email, SMS, Admin

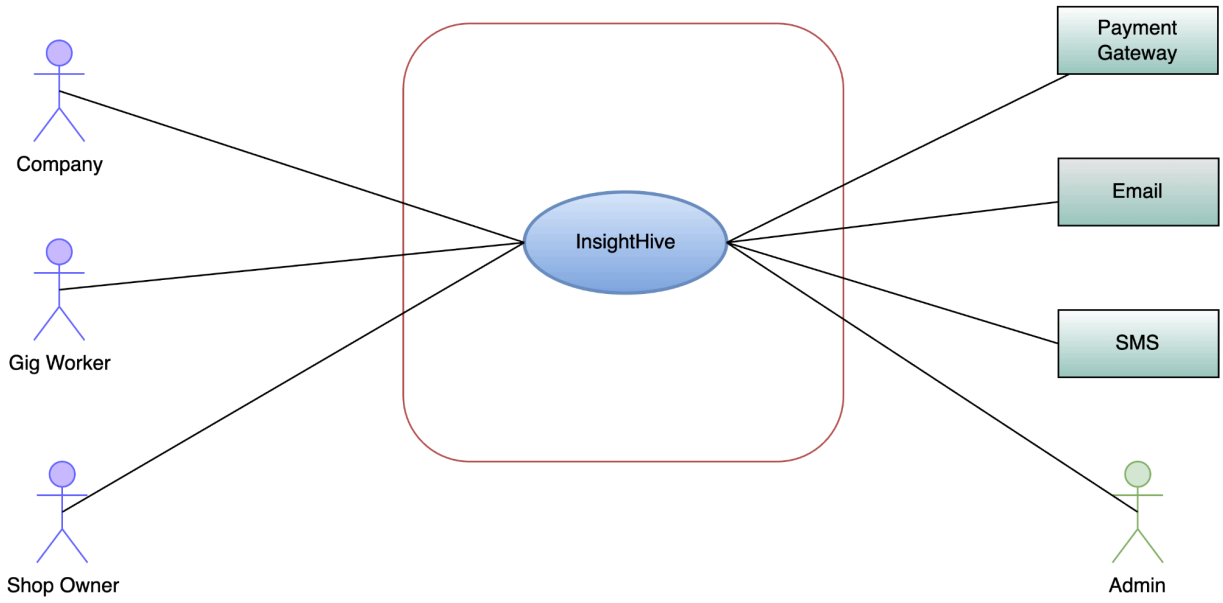

Figure 01: InsightHive App

Level 1:

Use Case ID: 1
Name: InsightHive-Detailed
Primary Actor: Gig Worker, Company, Shop Manager
Secondary Actor: Payment Gateway, Email, SMS, Admin

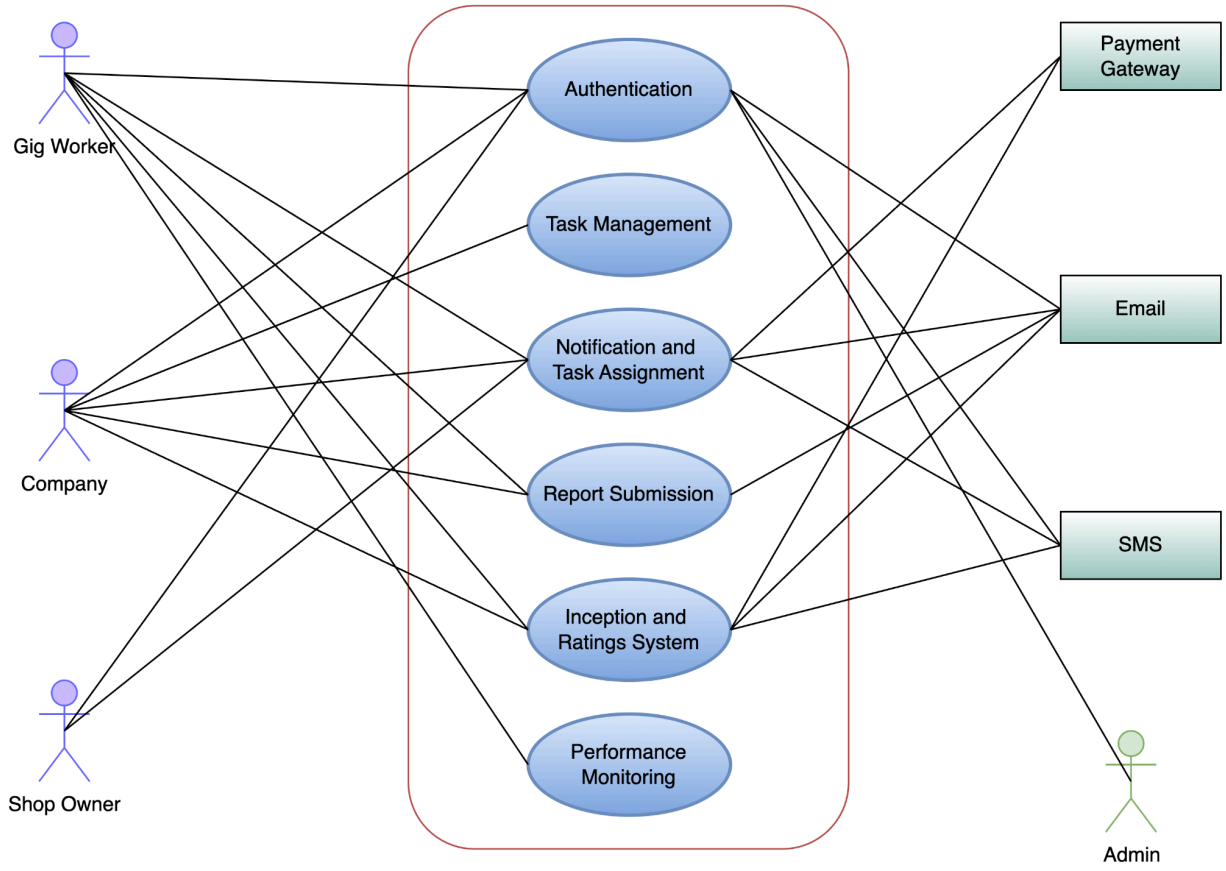

Figure 02: InsightHive-Detailed

Level 1.1: Authentication

Use Case ID: 1.1
Name: Authentication
Primary Actor: Gig Worker, Company, Shop Manager
Secondary Actor: Email, SMS, Admin

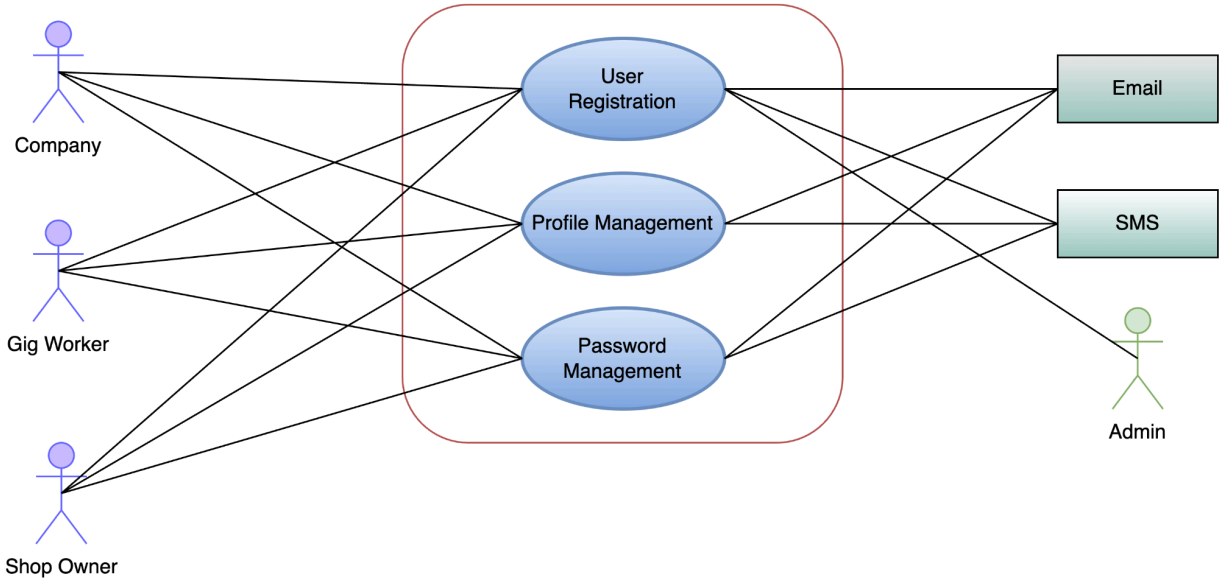

Figure 03: Authentication

Description

Company: Registers to create tasks and view insights.
Gig Worker: Registers to access tasks, submit reports, and earn incentives.
Shop Manager: Register to add shop to the system and verify gig workers at shop.

The user selects the "Register" option on the app.The system prompts the user to choose their role (Business User, Gig Worker or Shop Manager).The user enters required information.The system validates the provided information and creates an account.The user receives a confirmation message, and the system sends a verification email (or SMS) to confirm the registration.The user confirms the registration by entering the unique verification code.The system verifies the account and grants access to the platform.If the user enters invalid or incomplete information, the system prompts them to correct the details.If the email is already registered, the system notifies the user and suggests login or password recovery.

The required information:

Gig Worker: Full name, email, password, current address.
Business User: Company name, email, password, company details.
Shop Manager : Shop name, location, email, password, shop details.

Level 1.1.2 : Profile Management
Use Case ID: 1.1.2
Name: Profile Management
Primary Actor: Gig Worker, Company, Shop Manager
Secondary Actor: Email

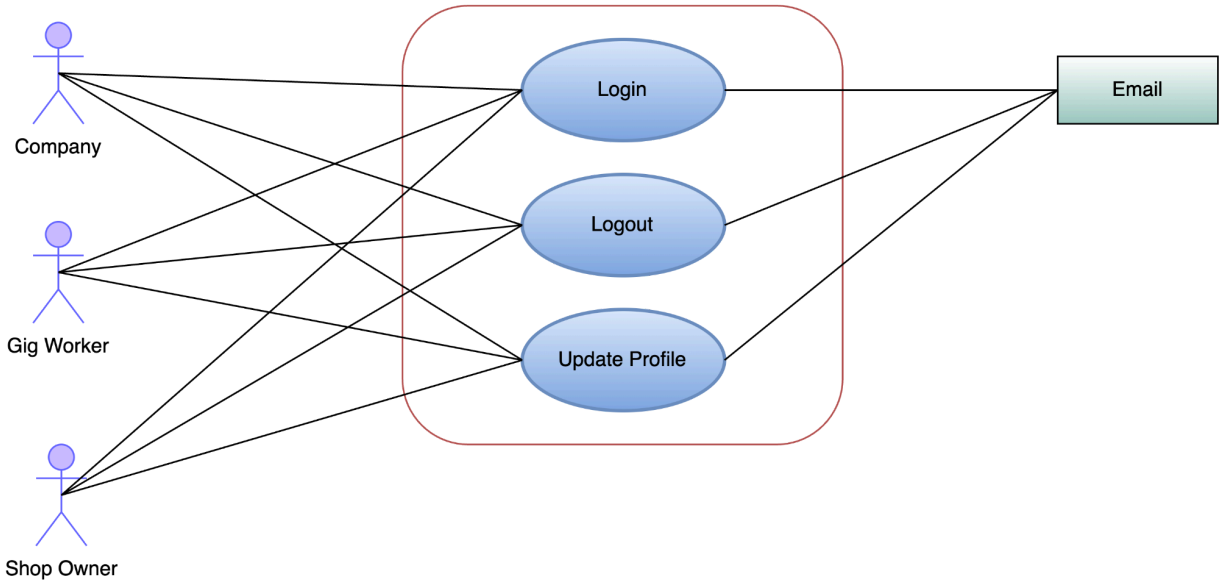

Figure 04: Profile Management

Company: Logs in to create tasks, and evaluate reports.
Gig Worker: Logs in to access tasks, submit reports and view incentives.
Shop Manager: Logs in to verify gig workers at shop.

The user selects the "Login" option on the app.The system prompts the user to enter their registered email and password.The user submits their credentials.
The system verifies the credentials:

If valid, the user is granted access to the platform.
If invalid, the system prompts the user to reenter or reset their password.

➢ On successful login, the user is directed to their dashboard:

Business User Dashboard: Access to task creation and data insights.
Gig Worker Dashboard: Access to available tasks and profile.

➢ Alternative Flows:

Forgot Password: If the user forgets their password, they can select "Forgot Password" to receive an OTP via email.
Unverified Account: If the account is not verified, the system prompts the user to complete the verification process.

Level 1.1.3 : Password Management

Use Case ID: 1.1.3

Name: Password Management

Primary Actor: Gig Worker, Company, Shop Manager

Secondary Actor: Email, SMS

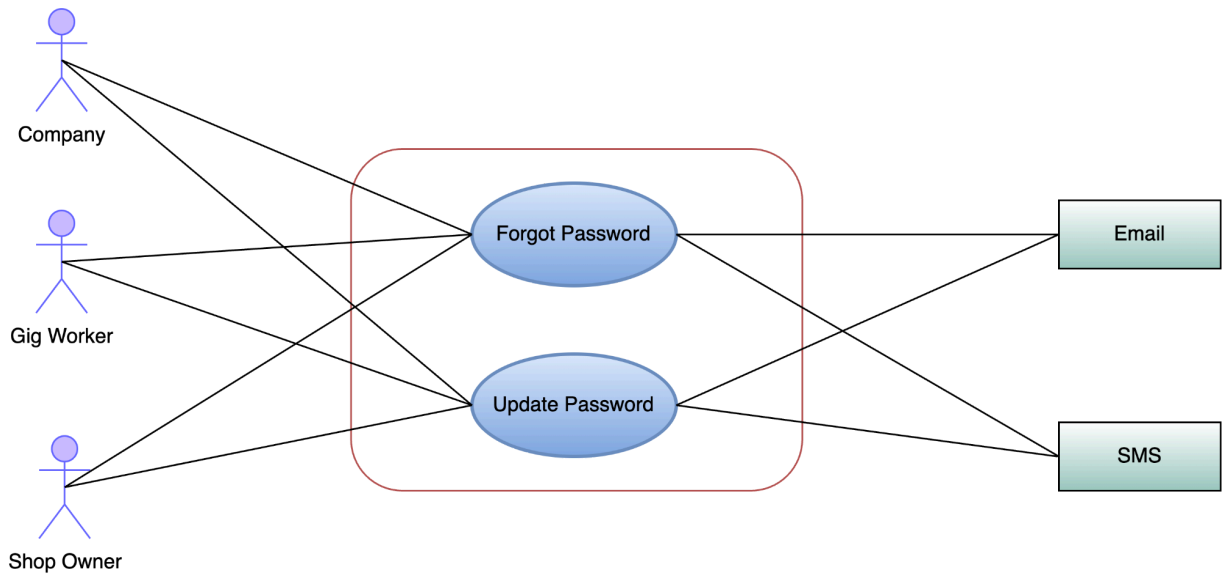

Figure 05: Password Management

Level 1.2 : Task Management

Use Case ID: 1.2

Name: Task Management

Primary Actor: Gig Worker, Company

Secondary Actor:Email, SMS

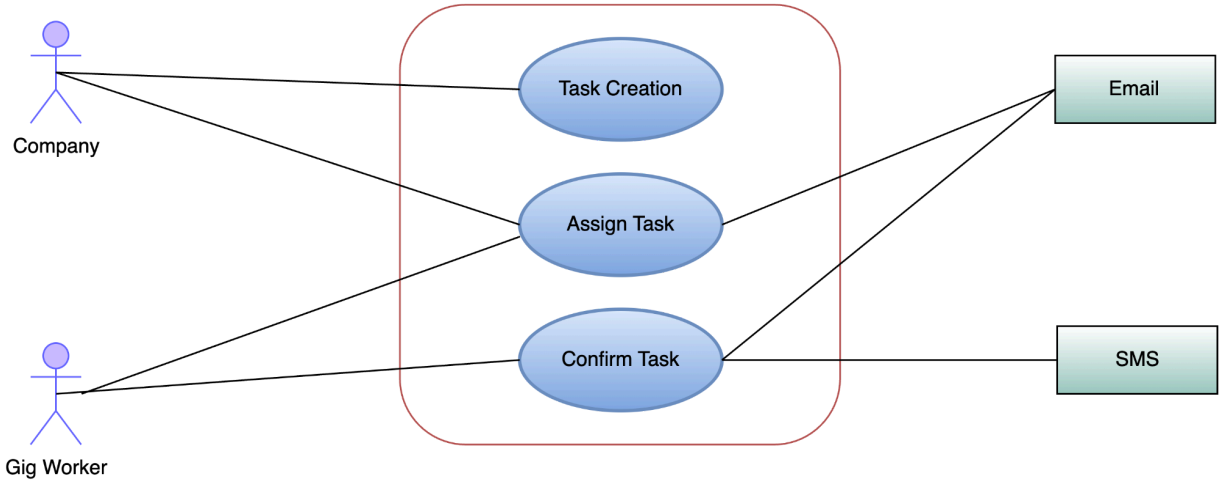

Figure 06: Task Management

The goal is to create tasks for gig workers in specific locations.The Company logs into the platform and navigates to the task creation page.They enter task details, such as location, requirements (e.g., text, photos,pdf), and deadline.The system lists available gig workers near the specified location.The Company selects the gig workers to assign task.Selected gig workers are notified of the new task.

Level 1.3: Notification & Task Assignment

Use Case ID: 1.3

Name: Notification & Task Assignment

Primary Actor: Gig Worker, Shop Manager

Secondary Actor: Payment Gateway, Email, SMS

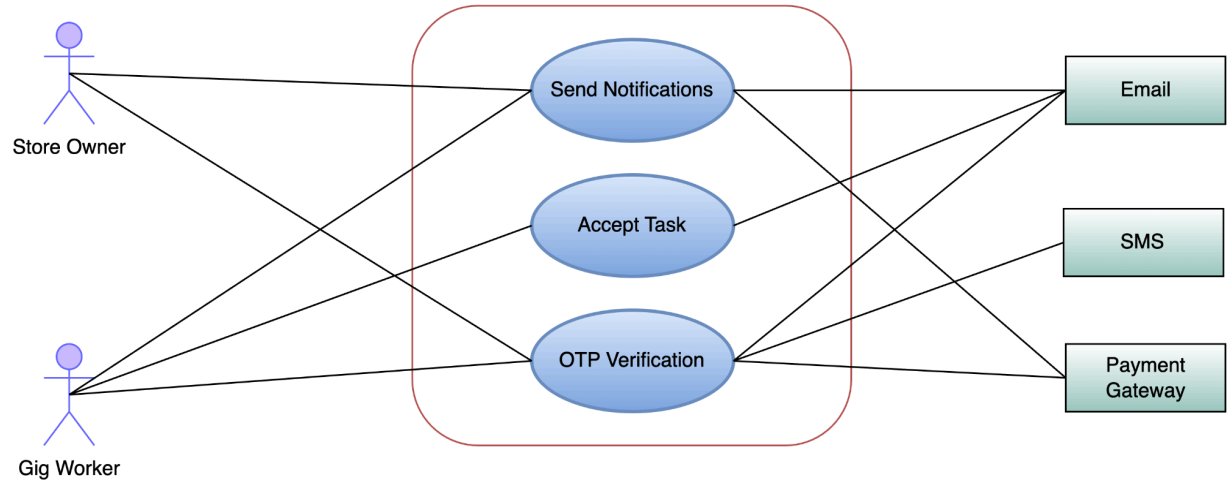

Figure 07: Notification And Task Assignment

The system sends notification to gig workers.The Gig Worker receives a notification of a new task available nearby.They view task details, including requirements and incentives. He confirms a task if he is interested in it. If only one worker is interested in the task, the company assigns the task to the worker.If multiple workers show interest in the same task, then the company selects a worker based on his previous performance. The system sends unique code to the gig worker and shop manager for confirmation.The Gig Worker arrives at the shop location.

He requests verification by presenting a unique code to the shop manager.The shop manager receives the unique code, verifies it, and confirms the gig worker's presence.

Level 1.4: Report Submission

Use Case ID: 1.4
Name: Report Submission
Primary Actor: Gig Worker
Secondary Actor: Email

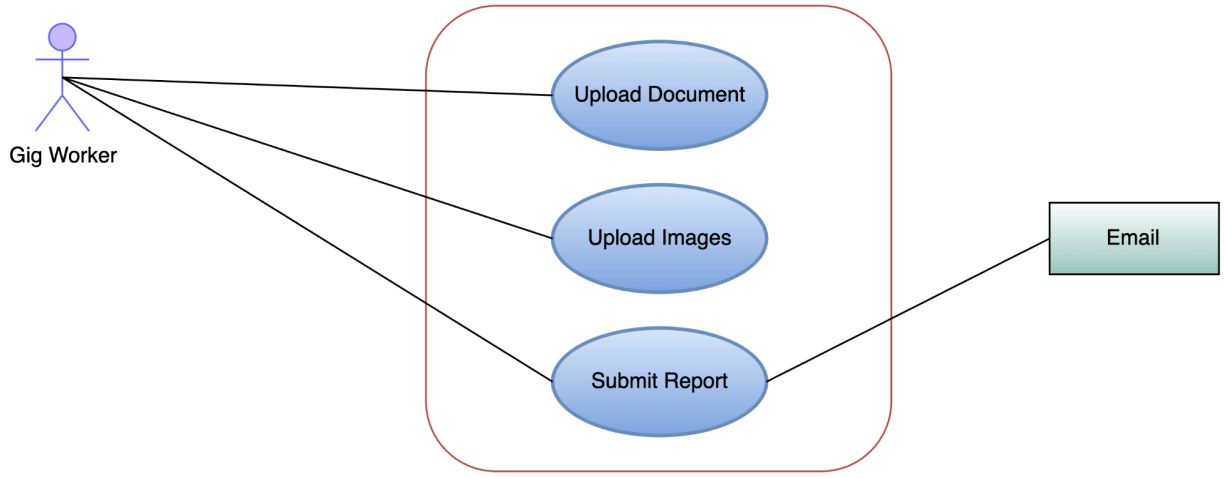

Figure 08: Report Submission

The Gig Worker uses the app to capture required data (e.g., photos, text, pdf). He fills out any additional details or observations as specified in the task. He submits the completed report through the platform.

Level 1.5: Incentive and Rating

Use Case ID: 1.5
Name: Incentives and Rating
Primary Actor: Gig Worker, Company
Secondary Actor: Email, SMS

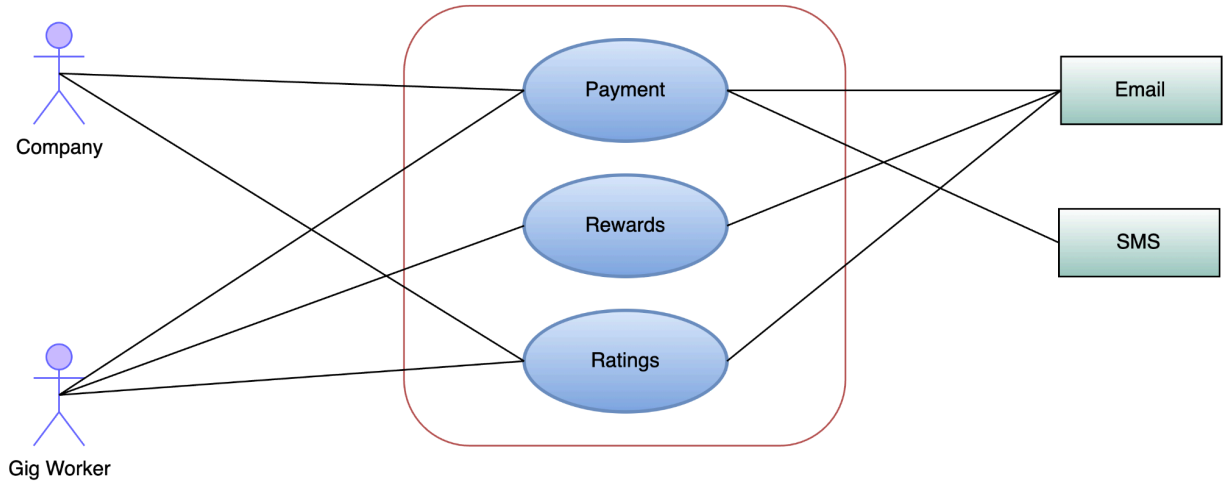

Figure 09: Incentive And Rating

The gig worker completes the task and submits the report. The Company rates the gig worker's performance based on quality and timeliness. The system updates the gig worker's earnings and rating. The gig worker can view their new rating and any incentives received in their profile.

Level 1.6: Performance Monitoring

Use Case ID: 1.6

Name: Performance Monitoring

Primary Actor: Gig Worker

Secondary Actor: Email

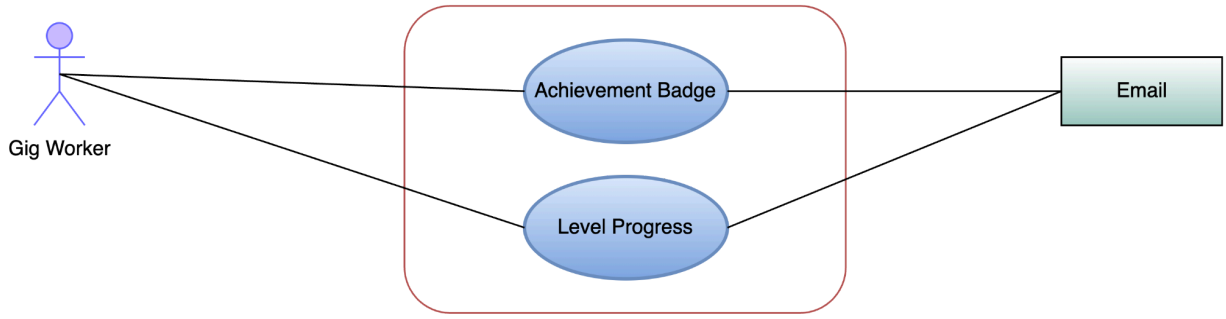

Figure 10: Performance Monitoring

The gig worker views the leaderboard to see their ranking among other workers.Points are awarded based on task completion and performance ratings.The gig worker sees their ranking, which encourages them to maintain or improve quality in future tasks.

2. Identifying Function Points

A **Function Point (FP)** is a **standard unit of measurement** for the **functionality** a software system provides to the user, based on **what the system does**, not **how it is built**.

The Five Function Types

Function Points are calculated by identifying and counting these five types of functions:

Function Type	Description	Example
External Inputs (EI)	Data coming into the system	User form submission
External Outputs (EO)	Data going out of the system	Reports, invoices
External Inquiries (EQ)	Requests for data (no update)	Search, lookups
Internal Logical Files (ILF)	Data the system maintains	Customer database
External Interface Files (EIF)	Data the system uses but doesn’t maintain	Imported product list

We use the client's perspective to identify functionality. The functions are categorized based on user-visible features and system interactions.

FID	Functionality of the Software, as described by Client	Function Type
F1	Multi-role User Registration (Company, Gig Worker, Shop Manager)	External Input
F2	User Authentication and Login System	External Input
F3	Password Reset and Recovery (OTP via Email/SMS)	External Input
F4	User Profile Management and Customization	External Input
F5	Task Creation and Configuration	External Input
F6	Task Assignment and Worker Selection	External Input
F7	Verification Code Generation and Distribution	External Output
F8	Task Notification System	External Output
F9	Worker Verification at Shop Location	External Input
F10	Photo and Text Data Collection	External Input
F11	Report Submission and Validation	External Input
F12	Task Review and Rating System	External Input
F13	Earnings Calculation and Update	External Output
F15	Leaderboard and Performance Tracking	External Inquiries
F16	Gamification System (Badges, Points, Levels)	External Output
F17	Real-time Notification Delivery (Email/SMS)	External Output
F18	Dashboard Analytics and Reporting	External Inquiries
F19	User Database Management	Internal Files
F20	Task Database Management	Internal Files
F21	Report Database Management	Internal Files
F22	Payment Transaction Records	Internal Files
F23	Performance and Rating Records	Internal Files
F24	Show Available Tasks to Workers	External Inquiries

F25	Company Task Management Interface	External Inquiries
F26	Worker Performance Analytics	External Inquiries

3. Unadjusted Function Points (UFP)

All features are not equally complex so in this step, we assign their complexity as a categorical estimate and weight them.

3.1 Assigning Complexity Categories

FID	Functionality	Complexity	Justification
F1	Multi-role User Registration	Complex	Multiple user types, validation, email verification
F2	User Authentication System and Login System	Average	Standard login with session management
F3	Password Reset/Recovery	Average	OTP generation and email/SMS integration
F4	Profile Management	Simple	Basic CRUD operations on user data
F5	Task Creation	Complex	Multiple parameters, location, deadline management
F6	Task Assignment	Complex	Worker matching, selection algorithms
F7	Verification Code Generation	Simple	Random code generation and distribution
F8	Task Notifications	Average	Multi-channel notification system
F9	Worker Verification	Average	Code matching and location verification
F10	Data Collection	Complex	Multiple file types, photo upload, text processing, pdf files
F11	Report Submission	Complex	File validation, quality checks, submission processing
F12	Task Review/Rating	Average	Review interface with rating system

F13	Earnings Calculation	Complex	Complex calculations based on performance and ratings
F15	Leaderboard System	Average	Ranking calculations and display
F16	Gamification Features	Complex	Badge system, point calculations, level progression
F17	Notification Delivery	Simple	Email/SMS sending
F18	Analytics Dashboard	Complex	Data aggregation, reporting, visualization
F19	User Database	Complex	Multi-role user management, relationships
F20	Task Database	Average	Task storage and retrieval
F21	Report Database	Average	Report storage with file management
F22	Payment Records	Average	Transaction history and tracking
F23	Performance Records	Simple	Rating and performance data storage
F24	Show Available Tasks	Average	Query and filter available tasks
F25	Task Management Interface	Average	Company dashboard for task management
F26	Performance Analytics	Complex	Advanced analytics and reporting

Weighting Factor Reference Table

This **Weighting Factor Reference Table** comes from the **Function Point Analysis (FPA)** method and was originally defined by **Allan Albrecht (IBM, late 1970s)** through empirical research.

Function Type	Simple	Average	Complex
Internal Files	7	10	15
External Interface Files	5	7	10
External Inputs	3	4	6
External Outputs	4	5	7
External Inquiries	3	4	6

3.2 UFP Calculation by Function Type

External Inputs

FID	Functionality	Complexity	Weight
F1	Multi-role User Registration	Complex	6
F2	User Authentication System	Average	4
F3	Password Reset/Recovery	Average	4
F4	Profile Management	Simple	3
F5	Task Creation	Complex	6
F6	Task Assignment	Complex	6
F9	Worker Verification	Average	4
F10	Data Collection	Complex	6
F11	Report Submission	Complex	6
F12	Task Review/Rating	Average	4
SUBTOTAL			**55**

Internal Files

FID	Functionality	Complexity	Weight
F19	User Database	Complex	15
F20	Task Database	Average	10
F21	Report Database	Average	10
F22	Payment Records	Average	10
F23	Performance Records	Simple	7
SUBTOTAL			**52**

External Outputs

FID	Functionality	Complexity	Weight
F7	Verification Code Generation	Simple	4
F8	Task Notifications	Average	5
F13	Earnings Calculation	Complex	7
F16	Gamification Features	Complex	7
F17	Notification Delivery	Simple	4
SUBTOTAL			**27**

External Inquiries

FID	Functionality	Complexity	Weight
F15	Leaderboard System	Average	4
F18	Analytics Dashboard	Complex	6
F24	Show Available Tasks	Average	4
F25	Task Management Interface	Average	4
F26	Performance Analytics	Complex	6
SUBTOTAL			**24**

Total UFP = 158

4. Degree of Influence (DI)

The **Degree of Influence (DI)** is a factor used in software estimation methods—especially in **Function Point Analysis (FPA)**—to adjust the raw function point count based on the overall complexity of the system.

The following 14 factors influence the implementation aspects of the functionalities, aside from their inheritance of these complexities. These factors are the additional technical requirements that those functionalities must fulfill.

1. Data Communication
2. Distributed Functions
3. Performance
4. Heavily Utilized Hardware
5. High Transaction Rates
6. On-line Data Entry
7. Online Updating
8. End-user Efficiency
9. Complex Computations
10. Reusability
11. Ease of Installations
12. Ease of Operations
13. Portability
14. Maintainability/Facility Change

We will now estimate the influence of these factors by assigning a score between 1-5.

Technical Complexity Factors (Scale 0-5)

Factor	Score	Rationale
Data Communication	3	Medium - Real-time notifications, API communications
Distributed Functions	2	Low-Medium - Simplified architecture
Performance	3	Medium - Standard performance requirements
Heavily Utilized Hardware	2	Low - Standard mobile/web hardware
High Transaction Rates	3	Medium - Moderate concurrent users
On-line Data Entry	5	High - Continuous data input from all user types
Online Updating	4	High - Real-time status updates

End-user Efficiency	4	High - Critical for gig worker productivity
Complex Computations	2	Low - Simplified algorithms
Reusability	3	Medium - Component-based architecture
Ease of Installation	4	High - Multi-platform deployment
Ease of Operations	3	Medium - Standard admin interfaces
Portability	4	High - Cross-platform requirements
Maintainability/Facility Change	3	Medium - Standard maintenance requirements

Total DI = 46

5. Technical Complexity Factor (TCF)

Technical Complexity Factor (TCF) is a measure of the **technical difficulty of a software project**. It is used in software estimation to adjust the estimated size of the project based on the technical considerations involved. The TCF is typically calculated by assigning a score between 0 and 5 to each of a set of technical factors, such as the complexity of the system architecture, the use of new or unfamiliar technologies, and the level of integration required with other systems. The scores are then multiplied by weighted values for each factor, and the total of all calculated values is the TCF.

TCF is a key factor in the Use Case Points (UCP) estimation technique. UCP is a size estimation method that is based on the number and complexity of use cases in a system. The TCF is used to adjust the UCP estimate to account for the technical difficulty of the project.

Formula: ***TCF = 0.65 + 0.01 × DI***

This formula is derived from the foundational work of Gustav Karner, who developed the Use Case Points (UCP) estimation method in 1993. This specific formula calculates the Technical Complexity Factor (TCF)

Calculation: TCF = 0.65 + 0.01 × 46 = **1.11**

The TCF score of 1.11 indicates that the project is relatively simple. This is because the TCF score is below 1.2, which is the threshold for low complexity projects.

6. Function Points Calculation

Function Points (FP) is a unit of measurement for software size. It is used to estimate the cost, effort, and schedule of software development projects. FP is calculated by counting the number of elementary processes in a software system. Elementary processes are the smallest units of functionality that are meaningful to the user.

FP is a widely used software size estimation method, and it is supported by a number of industry standards bodies, including the International Function Point User Group (IFPUG) and the Object Management Group (OMG).

$$FP = UFP \times TCF$$

Calculation: FP = 158 × 1.11 = 175.38
Final Function Points = 175

The FP count can be used to estimate the cost, effort, and schedule of the software project by using historical data from previous projects. For example, if the average cost of developing a Function Point is $100, then the estimated cost to develop a software project with an FP count of 175.38 would be $17538.

However, we will use standard COCOMO models for cost estimation due to their reliability.

7. Estimate Lines of Code (LoC) from Function Points

$$LOC = Language\ Factor * FP$$

Language Comparison

Language	Language Factor	Function Points	Lines of Code

Assembly	320	175	56,000
C	128	175	22,400
Java	55	175	9,625
Python	20	175	3,500
JavaScript	**31**	**175**	**5,425**

Selected Technology: JavaScript (Node.js+Express.js)
Estimated Total: 5.4 KLOC+

8. COCOMO Cost Estimation

The Constructive Cost Model was introduced by Dr. Barry Boehm. Software projects under COCOMO basic model strategies are classified into 3 categories, organic, semi-detached, and embedded

A. Organic Mode
A software project is said to be an organic type if -

- The project is small and simple.
- The project team is small with prior experience.
- The problem is well understood and has been solved in the past.
- Requirements of projects are not rigid, such a mode example is payroll processing system.

B. Semi-Detached Mode
A software project is said to be a Semi-Detached type if -

- The project has complexity.
- The project team requires more experience, better guidance, and creativity.
- The project has an intermediate size and has mixed rigid requirements such a mode example is a transaction processing system which has fixed requirements.
- It also includes the elements of organic mode and embedded mode.
- Few such projects are- Database Management System(DBMS), new unknown operating system, difficult inventory management system.

C. Embedded Mode
A software project is said to be an Embedded mode type if -

- A software project has fixed requirements of resources.

- Product is developed within very tight constraints.
- A software project requiring the highest level of complexity, creativity, and experience requirement falls under this category.
- Such mode software requires a larger team size than the other two models.

Model Selection: Organic Mode

Justification for Organic Mode:

- Project is moderately sized with well-understood patterns
- Small team with experienced developers
- Gig economy platform patterns are well-established
- Requirements are flexible in implementation details
- Similar platforms have been built before

COCOMO Parameters

Mode	a	b	c	d
Organic	**2.4**	**1.05**	**2.5**	**0.38**
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Cost Calculations

According to the basic COCOMO estimation model-

Metric	Formula	Calculation	Result
Effort	a × (KLOC)^b	2.4 × (5.4)^1.05	**13.8 Person-Months**
Duration	c × (Effort)^d	2.5 × (13.8)^0.38	**6.8 Months**
Team Size	Effort ÷ Duration	13.8 ÷ 6.8	**2.02 People**

Project Estimates Summary

Key Metrics

- **Function Points:** 175
- **Lines of Code:** 5.4 KLOC
- **Development Effort:** 13.8 Person-Months
- **Project Duration:** 6.8 months
- **Team Size:** 2 developers

Project Characteristics

- **Development Model:** Organic
- **Technology Stack:** JavaScript ecosystem (Node.js+Express.js)
- **Delivery Timeline:** ~7 months with 2-person team

Function Distribution

- **External Inputs:** 55 points (35%)
- **Internal Files:** 52 points (33%)
- **External Outputs:** 27 points (17%)
- **External Inquiries:** 24 points (15%)

References

[1] Software Engineering: A Practitioner's Approach by Roger Pressman
[2] Function Point Analysis: Measurement Practices for Successful Software Projects
[3] COCOMO II Model Definition Manual by Barry Boehm
[4] QSM Function Point Languages Table (gearing factors)

[5] https://ifpug.org/ifpug-standards/fpa (function point analysis)

[6] https://en.wikipedia.org/wiki/COCOMO (COCOMO parameter table)